

CSCI 12532

Computer Architecture and Design

Lecture 10

Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

Contents

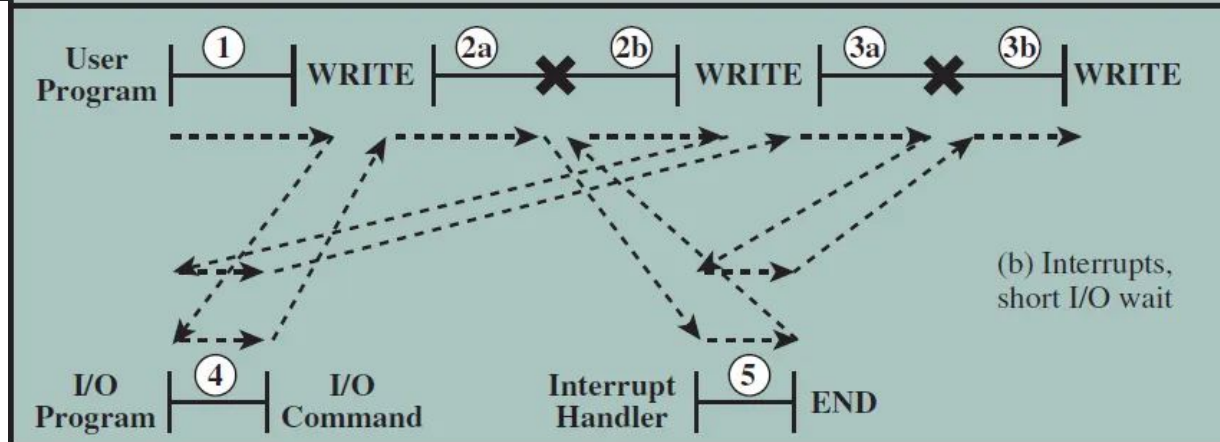
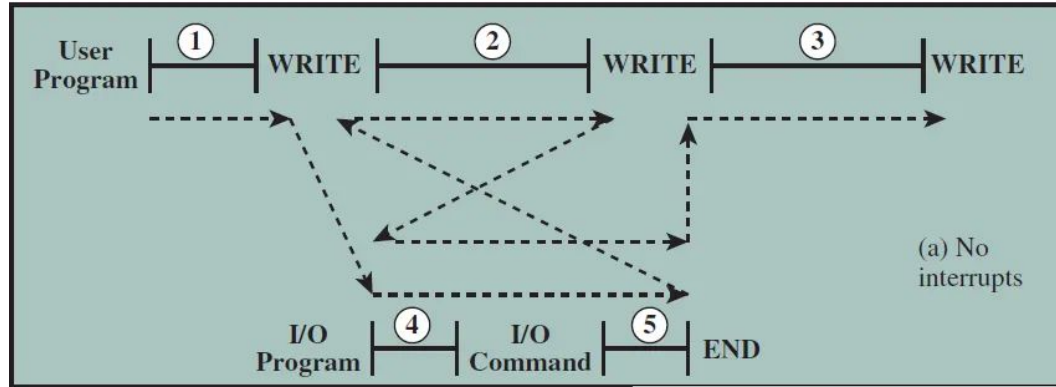
1. An Interrupt?
2. Types of interrupts
3. Handling Interrupts

1. AN INTERRUPT

Suppose that the processor is transferring data to a I/O device (let's say a printer) using the instruction cycle. However, most external devices are much slower than the processor. So after each write operation, the processor has to pause and remain idle until the printer catches up.

The length of this pause may be equivalent to hundreds or even thousands of instruction cycles that could have occurred. Clearly, this is a very wasteful use of the processor.

Interrupt vs. No interrupt



What is an interrupt?

An interrupt is a signal or request that prompts the processor to temporarily halt its current operations and execute a specific task or service. It allows the CPU to respond to events or conditions that require immediate attention, such as input/output operations or hardware malfunctions. Interrupts ensure efficient system performance by enabling the processor to handle critical tasks without continuously polling for events.

What is polling?

Polling is a method where a device continuously checks the status of another device to determine if it needs attention. This involves sending requests or queries to the device, waiting for a response, and repeating this process in a loop until the desired condition or data is obtained.

- **Simplicity:** It is a simpler mechanism to implement and provides straightforward control over the process.
- **Compatibility:** Polling can work on systems that do not support interrupt-driven mechanisms, making it widely applicable across various platforms.
- **Predictability:** Since the system checks for data at regular intervals, the behavior is predictable and can be fine-tuned for specific performance requirements.

Disadvantages of polling

Resource Intensive: Constantly checking for data or events consumes system resources, including CPU cycles, leading to inefficiency and wasted power.

Latency: There can be a delay between the occurrence of an event and the system detecting it, especially if the polling interval is long, leading to higher latency in event processing.

Complexity in Real-Time Systems: In real-time applications where immediate responses are critical, polling may not be suitable.

When to Use Polling vs Interrupts

Polling is ideal for events that occur at consistent and predictable intervals, allowing the system to check for updates periodically. It is also suitable for simple applications where a constant check for events doesn't significantly impact system performance and where a straightforward implementation is preferred.

Interrupts, on the other hand, are more ideal for when immediate responses to external events are critical, such as real-time systems. They are also best for event-driven applications where variable event timing is common.

Interrupt vs. Polling

Feature	Interrupts	Polling
Definition	Hardware/software signals trigger immediate CPU response.	Repeatedly checks a port/sensor at fixed intervals.
Efficiency	Efficient (only activates when needed).	Inefficient (checks even if no change).
Timing	Less predictable (depends on event).	Predictable (fixed intervals).
Complexity	Harder to implement.	Simple to code.
CPU Usage	Saves CPU resources.	Wastes cycles on unnecessary checks.
Process Flow	1. Pauses current task, 2. Saves return address (stack), 3. Runs interrupt code, 4. Resumes original task.	Continuously loops checks.
Use Case	Ideal for event-driven systems (e.g., I/O devices, timers)	Suitable for predictable systems
Hardware Support	Built-in timers & dedicated interrupt lines.	Manual implementation.

2. TYPES OF INTERRUPTS

1. Hardware Interrupts

Generated by hardware devices.

Causes:

- Hardware failure (e.g., power issue, device error).
- I/O operation completion (e.g., keyboard input, disk read/write).

Subtypes:

- Internal Interrupts (from CPU, e.g., divide-by-zero).
- External Interrupts (from peripherals, e.g., mouse click).

Hardware Interrupt is An electronic signal sent to the CPU from external hardware (e.g., keyboard, mouse, disk controller).

How It Works?

Example:

- Pressing a keyboard key → Triggers interrupt → CPU reads input.
- Moving a mouse → Triggers interrupt → CPU updates cursor position.

Types

- Maskable Interrupts
 - Can be disabled/ignored by the CPU (e.g., via CLI instruction).
 - Used for low-priority tasks (e.g., I/O operations).
- Non-Maskable Interrupts (NMI)
 - Cannot be disabled/ignored by the CPU.
 - Used for critical events (e.g., hardware failure, power loss).

2. Software Interrupts

Triggered by software (usually in user mode).

Causes:

- Program termination.
- System service requests (e.g., screen output, printer access).
- Higher priority than hardware interrupts.

Subtypes:

- Normal Interrupts (expected, e.g., system calls).
- Exception Interrupts (unexpected errors, e.g., illegal instruction).

How It Works?

Example:

- Divide-by-zero → Aborts calculation/displays error
- System calls → Switches to kernel mode for OS services

Key Characteristics

Works like a subroutine call (saves state, jumps to handler, returns)

Used for:

- Error handling (e.g., exceptions)
- Controlled transitions (e.g., user mode → kernel mode)

Classes of Interrupts

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation, request service from the processor, or to signal a variety of error conditions.
Hardware Failure	Generated by a failure such as power failure or memory parity error.

Interrupt Triggering Methods

- Edge Triggering: An interrupt is generated when a signal transitions from one state to another, such as from low to high (rising edge) or high to low (falling edge). Edge triggering is useful for detecting changes in state.
- Level Triggering: An interrupt is generated when a signal is at a specific level, either high or low, for a certain period. Level triggering is beneficial for detecting steady-state conditions.
- Software Triggering: Interrupts can also be initiated by software through specific instructions or commands. This method allows greater flexibility and control within the software environment.
- Priority Triggering: Some systems implement priority-based triggering, where higher-priority interrupts can preempt lower-priority ones. This ensures that critical tasks are handled promptly.

3. HANDLING INTERRUPTS

Whenever an interrupt occurs, it causes the CPU to stop executing the current program. Then, comes the control to interrupt handler or interrupt service routine.

These are the steps in which ISR handles interrupts. These are as follows.

Step 1 : When an interrupt occurs let assume processor is executing i 'th instruction and program counter will point to the next instruction $(i+1)$ th.

Step 2 : When an interrupt occurs the program value is stored on the process stack and the program counter is loaded with the address of interrupt service routine.

Step 3 : Once the interrupt service routine is completed the address on the process stack is popped and placed back in the program counter.

Step 4 : Now it executes the resume for $(i+1)$ th line.

Interrupt Handler

- Acts as a queue/pool of interrupts waiting for CPU processing
- Manages prioritization and order of interrupt servicing

Interrupt Service Routine (ISR)

- A dedicated subroutine executed when an interrupt occurs
- Each interrupt has:
 - A unique ISR (specific handling code)
 - A fixed memory address storing the ISR location

Interrupt Vector Table (IVT)

- A predefined memory table containing addresses of all ISRs
- Functions:
 - Maps interrupt types → corresponding ISR
 - Ensures rapid CPU response (direct jump to ISR)

Types of Interrupt Handlers

1. First Level Interrupt Handler (FLIH) is a hard/fast interrupt handler. FLIHs are designed for maximum speed and minimal processing to prevent system jitter. It is the OS component that first responds to a hardware interrupt. It performs critical, time-sensitive tasks with interrupts disabled, quickly clears the hardware flag, and schedules a Second-Level Interrupt Handler (SLIH) for longer tasks
2. Second Level Interrupt Handler (SLIH) is a soft/slow interrupt handler. SLIHs run asynchronously as standard threads with lower priority, allowing the CPU to re-enable interrupts and prevent system delays. a deferred mechanism in operating systems designed to complete lengthy or complex interrupt processing that is deferred by the First-Level Interrupt Handler (FLIH)

Interrupt Latency

Interrupt latency is the time that elapses between the occurrence of an interrupt and the execution of the first instruction of the interrupt service routine (ISR) that handles the interrupt.

It is a measure of the system's ability to respond to external events in a timely manner.

The shorter the interrupt latency, the more responsive the system will be.

Interrupt latency is expressed in core clock cycles.

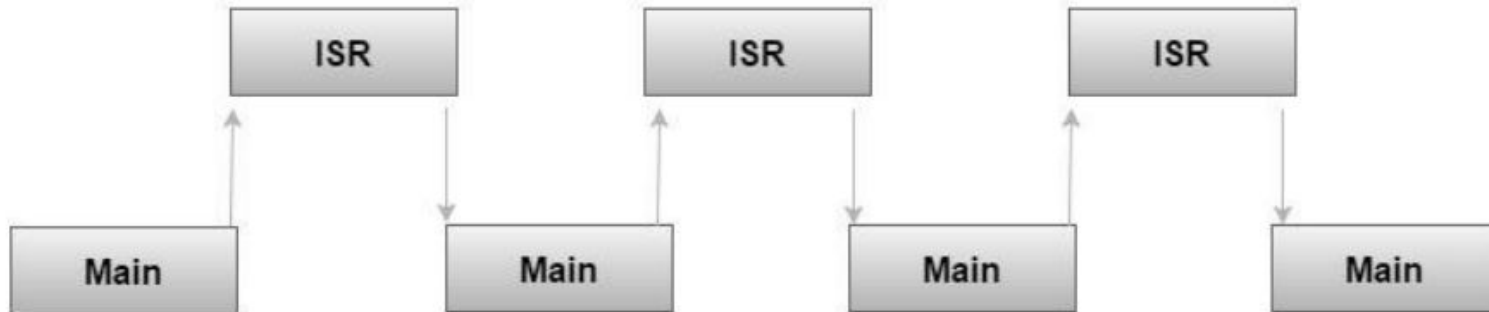
Program Execution without Interrupts

Time →



Program Execution with Interrupts

Time →



ISR : Interrupt Service Routine

Benefits of Interrupts

- **Efficiency:** By using interrupts, the CPU can perform other tasks instead of constantly polling for events. This increases overall system efficiency and allows for better utilization of processing resources.
- **Responsiveness:** Interrupts enable the system to respond quickly to external events. This is particularly important in real-time applications where delays can lead to system failures or degraded performance.
- **Modularity:** Interrupts support modular design by allowing different parts of a system to handle specific events independently. This separation of concerns can simplify development and maintenance.
- **Power Saving:** In low-power systems, interrupts can help conserve energy by allowing the CPU to enter low-power modes when idle and wake up only when an interrupt occurs, reducing overall power consumption.

Thank You

30.06.2026